*Name: Gowtham Prasad*

# CS403/503 Programming Languages
## Spring 2025
Assignment #3

1. Problem 13 of Chapter 7 on Page 327

Answer:

```
int fun(int* k) {
    *k += 4;
    return 3 * (*k) - 1;
}
void main() {
    int i = 10, j = 10, sum1, sum2;
    sum1 = (i / 2) + fun(&i);
    sum2 = fun(&j) + (j / 2);
}
```

a. (left → right): sum1 = 46, sum2 = 48
b. (right → left): sum1 = 48, sum2 = 46

2. Programming Exercise 1 of Chapter 8 on Page 362

Answer:

```
k = (j + 13) / 27
loop:
        if k > 10 then goto out
        k = k + 1
        i = 3 * k - 1
        goto loop
out: . . .
```

C++:

```
int i = 0, j = 0;
for (int k = (j + 1) / 27; k =< 10; k++) {
   i = 3 * k - 1;
}
```

Python:

```
i, j = 0, 0
k = (j + 1) / 27

while k <= 10:
   k += 1
   i = 3 * k - 1
```

Ruby:

```
i, j = 0, 0
k = (j + 1) / 27

while k <= 10
   k += 1
   i = 3 * k - 1
   end
end
```

3. Programming Exercise 3 of Chapter 8 on Page 363

Answer:

C++:

```
switch (k) {
        case 1: case 2:
```

```
                j = 2 * k - 1;
                break;
            case 3: case 5:
                j = 3 * k + 1;
                break;
            case 4:
                j = 4 * k - 1;
                break;
            case 6: case 7: case 8:
                j = k - 2;
                break;

            default:
                break;
        }
```

Python:

```
        match k:
            case 1 | 2:
                j = 2 * k - 1
            case 3 | 5:
                j = 3 * k + 1
            case 4:
                j = 4 * k - 1
            case 6 | 7 | 8:
                j = k - 2
            case _:
                pass
```

Ruby:

```
case k
when 1, 2
   j = 2 * k - 1
when 3, 5
   j = 3 * k + 1
when 4
   j = 4 * k - 1
when 6, 7, 8
   j = k - 2
else
   # do nothing
end
```

4. Programming Exercise 5 of Chapter 8 on Page 363

Answer: C++

```
int n;
   cin >> n;
   int x[n][n];
   for (int i = 0; i < n; i++) {
      bool found = true;
      for (int j = 0; j < n; j++) {
         if (x[i][j] != 0) {
            found = false;
            break;
         }
      }
      if (found) {
```

*Name: Gowtham Prasad*

```
            cout << "First all-zero row is: " << i + 1 << endl;
            break;
        }
    }
```

In my version of the code, I aimed to improve readability and maintainability by replacing the goto statement with structured control flow using a boolean flag and a break statement. Unlike the original goto-based code, which disrupts the natural flow of execution by jumping to a labeled statement, my implementation keeps the logic within conventional loop structures, making it easier to follow. By using the found flag, I explicitly track whether a row contains only zeros, eliminating the need for abrupt jumps and making the code's intent clearer. Additionally, this approach enhances maintainability, as modifying or extending the logic becomes more straightforward without the complications that goto can introduce. While my version is slightly more verbose due to the boolean flag, I believe this trade-off is worth it for the added clarity and structure. Overall, my approach aligns with modern programming best practices, making the code easier to read, modify, and debug.